

1. Członkowie zespołu i macierz kompetencji

Macierz kompetencji zespołu:

Członek \ Kompetencja	Kiryl Baranouski	Damian Kantor	Agata Ślusarczyk
Python	+	+	+
HTML	+	+	+
CSS	+	+	+
React	-	+	+
Bazy danych	+	+/-	+/-
Migracje	+	-	-
Git	+	+	+
Organizacja zespołu	+	+	+

2. Pytania uszczegóławiające projekt

Pytanie	Odpowiedź
Jakie wydarzenia będą raportowane?	Raportowane będą poprawne (data, godzina) i niepoprawne wejścia (nierozpoznana twarz. powód odmowy - nie powiązany z twarzą, data, godzina)
Jakie będą uprawnienia u administratora?	Dodawanie pracownika, uzupełnienie danych pracownika + zdjęcie/wideo, aktualizacja danych, nadawanie uprawnienia do wejścia, sprawdzenie raportu dla pracownika, generowanie QR, generowanie raportów
Jaką liczbę pracowników podtrzymuje baza?	Nie mniej 20 pracowników
Na ile dokładny jest system skanowania twarzy?	90% skuteczności
Jaki jest czas przetwarzania twarzy systemem?	Do 5 sekund
Ile będą przechowywane raporty	6 miesięcy

3. Format danych wejściowych i ich struktury

System kontroli wejścia będzie przetwarzał różne źródła danych. Przykładowe formaty i struktury danych wejściowych:

Dane pracowników: dla każdego pracownika przechowujemy co najmniej:

- Id: unikalny identyfikator pracownika; imię i nazwisko (ciągły tekstowe).
- rola: rola użytkownika w systemie (np. "Pracownik" lub "Administrator").

- face_embedding: wektor cech twarzy (liczby zmiennoprzecinkowe, np. 128- lub 512-wymiarowy) – reprezentacja twarzy używana do rozpoznawania.
- qr_code_data: ciąg znaków (np. w formacie JSON) – dane zakodowane w kodzie QR (zwykle identyfikator pracownika oraz data ważności przepustki).
- valid_to: data – data wygaśnięcia upoważnienia pracownika do wejścia (koniec ważności przepustki).

Dane z kodu QR: ciąg znaków odczytany z przepustki. Kod QR zawiera informacje o pracowniku i okresie ważności. Do odczytu używany jest moduł OpenCV – klasa cv2.QRCodeDetector służy do detekcji i dekodowania kodów QR bezpośrednio z obrazu wideo. Dane z QR mogą być zapisane w bazie danych (np. w polu tekstowym lub JSON w tabeli pracowników).

Raporty i zdarzenia: każdy incydent wejścia jest zapisywany jako rekord raportu. Struktura pojedynczego rekordu raportu wejścia zawiera m.in.:

- report_id: unikalny identyfikator zdarzenia (raportu).
- employee_id: identyfikator pracownika powiązanego z wejściem.
- timestamp: znacznik daty i czasu zdarzenia.
- access_granted: wartość logiczna (True/False) – informacja, czy dostęp został przyznany.
- notes: tekstowe uwagi (np. powód odmowy wejścia, jeśli wystąpiła).

Dane raportów mogą być przechowywane w relacyjnej bazie danych (SQLite) w tabeli logów wejść. Zgodnie z założeniami, logi zdarzeń będą przechowywane przez 6 miesięcy, po czym najstarsze wpisy mogą być archiwizowane lub usuwane.

4. Model opisu systemu (tabela)

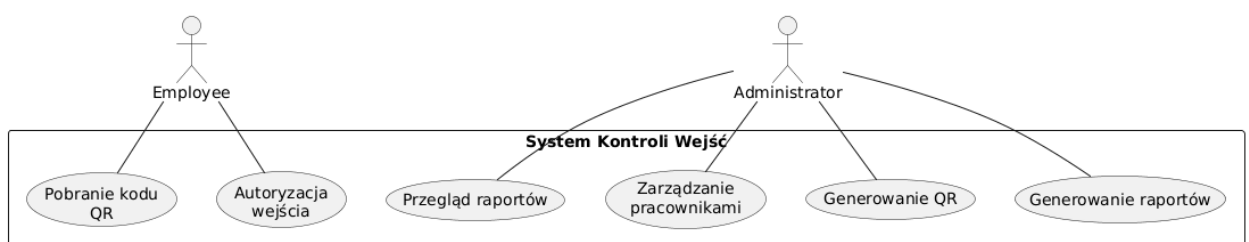
Opis funkcji systemu można przedstawić w tabeli ze scenariuszami działania:

Aktor	Dane wejściowe	Wyzwalacz	Odpowiedź systemu	Uwagi
Employee	id pracownika, dane z przepustki (QR), obraz twarzy z kamery	Skanowanie kodu QR oraz twarzy przy wejściu	Weryfikacja uprawnień; jeśli QR i twarz pasują: zezwól na wejście, dodaj wpis do rejestru; w przeciwnym razie zablokuj dostęp.	Detekcja twarzy i porównanie wektora cech twarzy (embedding) z wzorcem.

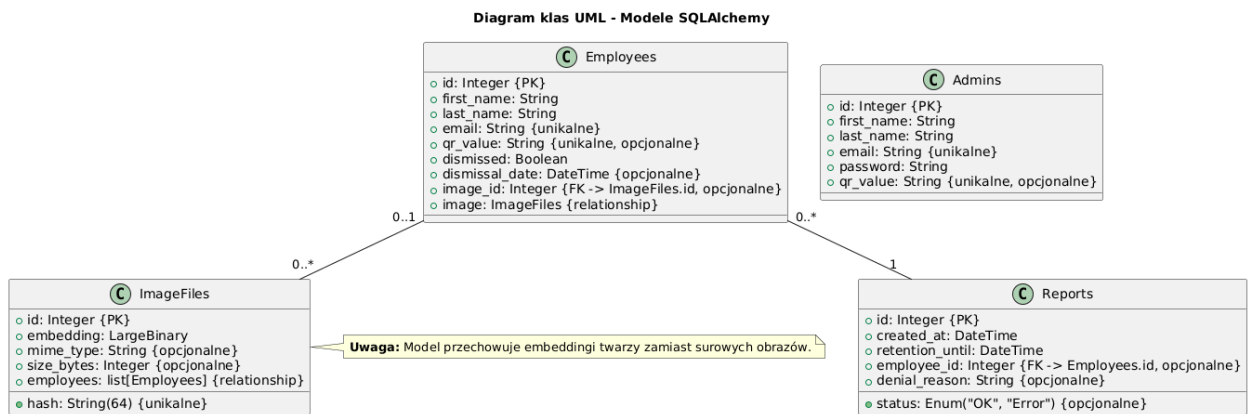
Admin	Lista pracowników, ich uprawnienia, zapisane raporty	Logowanie do panelu administracyjnego	Wyświetlenie panelu admina; umożliwienie zarządzania kontami (dodawanie/usuwanie pracowników, nadawanie uprawnień) oraz przeglądanie i generowanie raportów.	Dostęp zabezpieczony hasłem/2FA, role użytkowników.
System	Baza danych (listy uprawnionych osób, logi zdarzeń); harmonogram zadań (czas)	Codzienna zmiana daty lub żądanie zaplanowanego zadania	Generowanie zbiorczych raportów	Może działać w tle

Tabela prezentuje przypadki użycia aktorów w systemie. Dla każdego scenariusza wymieniono niezbędne dane, sposób wyzwolenia akcji oraz oczekiwaną reakcję systemu. Uwagi wskazują na dodatkowe wymagania (np. porównanie embeddingów twarzy).

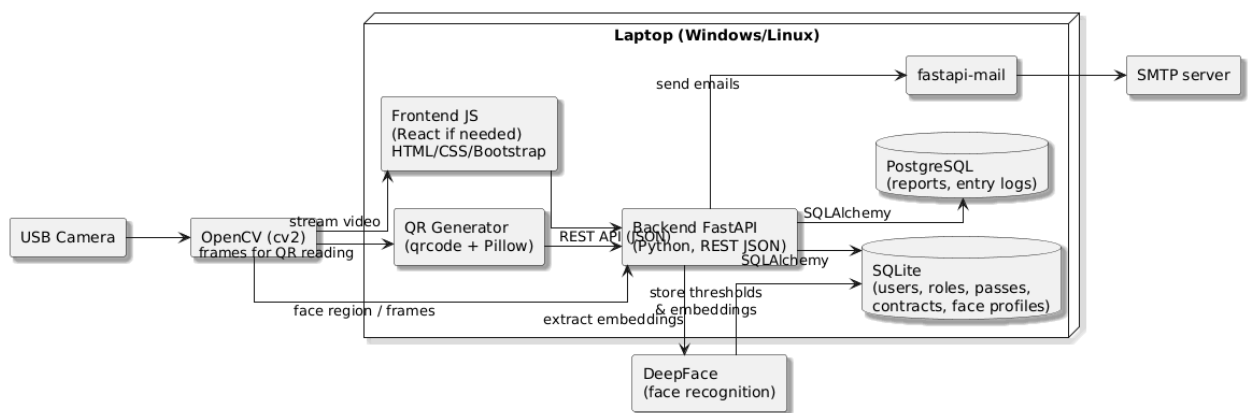
5. Diagramy UML



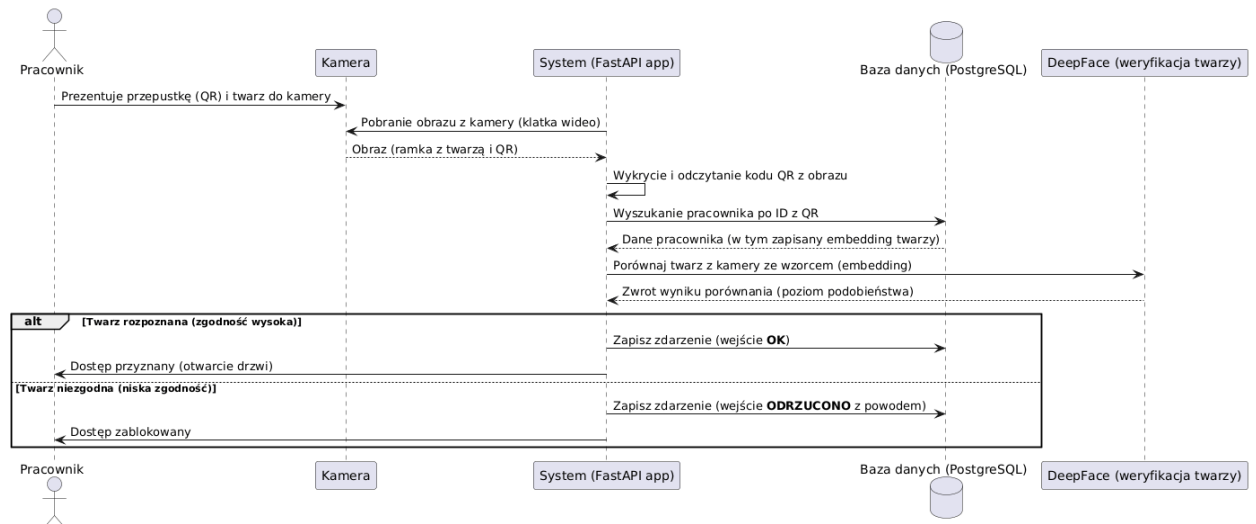
Rys. 1 – Diagram przypadków użycia (Use Case)



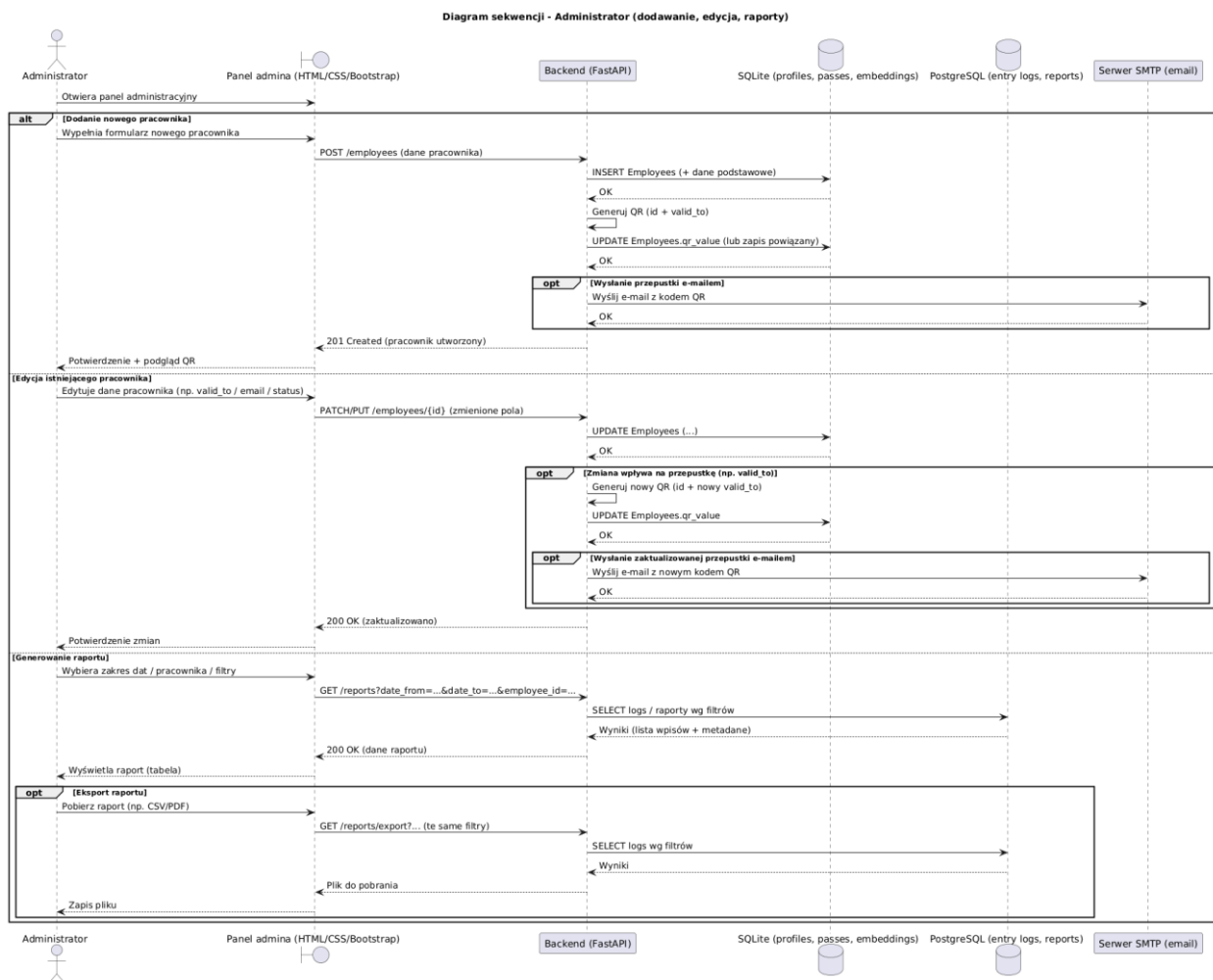
Rys. 2 – Diagram klas (Class Diagram)



Rys. 3 – Architektura systemu



Rys. 4 – Diagram sekwencji (Pracownik)



Rys. 5 – Diagram sekwencji (Administrator):

6. Opis działania systemu krok po kroku

Poniżej opisano szczegółowo przebieg działania systemu kontroli wejść, krok po kroku, zgodnie z aktualną implementacją i wymaganiami funkcjonalnymi. Proces autoryzacji został zaprojektowany tak, by obsłużyć do 20 pracowników i zajmować maksymalnie ok. 5 sekund na osobę przy ~90% skuteczności rozpoznawania twarzy.

Wczytanie kodu QR: Gdy pracownik podchodzi do wejścia, skanuje swoją przepustkę zawierającą kod QR (np. wydrukowaną kartę lub kod wyświetlony na telefonie) przed kamerą systemu. Oprogramowanie przy użyciu kamery USB przechwytuje obraz wideo i za pomocą modułu OpenCV (klasa cv2.QRCodeDetector) automatycznie wykrywa i dekoduje kod QR z klatki obrazu. Po odczytaniu zawartości kodu system uzyskuje unikalny identyfikator pracownika oraz datę ważności przepustki zakodowane w QR. Następnie aplikacja weryfikuje, czy taki identyfikator istnieje w bazie danych i czy przepustka jest nadal ważna (bieżąca data mieści się w okresie ważności).

Weryfikacja twarzy: Jeśli przepustka QR jest prawidłowa i przypisana do zarejestrowanego pracownika, system przystępuje do weryfikacji tożsamości osoby za pomocą obrazu z

kamery. Wykorzystywana jest biblioteka DeepFace do rozpoznawania twarzy – w pierwszym kroku następuje detekcja twarzy na obrazie z kamery (DeepFace korzysta z wbudowanego modelu detekcji, np. RetinaFace lub MTCNN). Z obrazu wejściowego wycinany jest fragment z twarzą pracownika, który następnie podlega normalizacji (np. wyrównanie względem oczu) w celu zapewnienia spójności.

Ekstrakcja cech twarzy (embedding): Wyodrębniona twarz przekazywana jest do algorytmu rozpoznawania twarzy z biblioteki DeepFace. Wykorzystywany jest pretrenowany model głębokiej sieci neuronowej (np. FaceNet lub ArcFace zaimplementowany w DeepFace) w celu przekształcenia obrazu twarzy na wektor liczb – tzw. embedding twarzy. Wektor embedding ma stałą długość (np. 128 lub 512 wymiarów) i jest unikalny dla charakterystycznych cech danej twarzy. Embedding referencyjny dla pracownika (wzorzec) został uprzednio obliczony i zapisany w systemie podczas rejestracji pracownika (w bazie danych jest przechowywany wektor cech twarzy powiązany z danym identyfikatorem pracownika).

Porównanie twarzy: System porównuje wygenerowany wektor cech z kamery z wektorem wzorcowym zapisanym dla danego pracownika w bazie danych. Porównanie odbywa się poprzez obliczenie metryki podobieństwa – najczęściej wykorzystywana jest odległość kosinusowa lub euklidesowa między dwoma wektorami. Biblioteka DeepFace zwraca wynik porównania, na podstawie którego aplikacja określa, czy jest to ta sama osoba. Jeśli podobieństwo przekracza ustalony próg, tożsamość posiadacza przepustki zostaje potwierdzona (match). Gdy podobieństwo jest zbyt niskie, oznacza to, że twarz nie pasuje do zapisanej – system traktuje to jako próbę wejścia przez nieuprawnioną osobę.

Decyzja i akcja systemu: W przypadku pozytywnej weryfikacji (poprawny kod QR + pasująca twarz) system odblokowuje wejście dla pracownika. Może to oznaczać wysłanie sygnału do drzwi lub bramki obrotowej celem ich otwarcia, a także wyświetlenie komunikatu potwierdzającego (np. zielone światło lub komunikat "Dostęp przyznany"). Jednocześnie system rejestruje to zdarzenie w bazie danych – dodawany jest nowy wpis raportu z oznaczeniem czasu, identyfikatorem pracownika oraz adnotacją, że dostęp został przyznany. W przypadku nieudanej weryfikacji (gdy twarz nie zostanie rozpoznana lub nie pasuje do osoby z przepustki) system natychmiast blokuje dostęp (np. nie otwiera drzwi, może zaświecić czerwone światło lub wyemitować sygnał dźwiękowy ostrzegawczy). Zdarzenie to również zostaje zapisane w systemie – z oznaczeniem, że była odmowa dostępu, wraz z powodem (np. "niezgodność twarzy z QR") i dokładnym czasem. Dzięki temu każda próba wejścia – zarówno poprawna, jak i niepoprawna – jest udokumentowana.

Rejestr zdarzeń i raportowanie: Wszystkie odnotowane próby wejścia trafiają do rejestru (logów) systemu. Administrator ma możliwość przeglądania tych zapisów na bieżąco oraz generowania raportów zbiorczych. Raporty mogą przybierać formę tabelaryczną (lista zdarzeń) lub zestawień statystycznych za wybrany okres. Na podstawie danych w logach można generować np. dzienne raporty pokazujące godziny wejść poszczególnych pracowników, łączny czas obecności, a także wykryte nadużycia (próby użycia cudzej karty

itp.). Raporty można filtrować per pracownik – np. administrator wybiera konkretnego pracownika, aby zobaczyć historię jego wejść i ewentualnych odmów dostępu.

Generowanie kodów QR i zarządzanie uprawnieniami: W panelu administracyjnym systemu dostępne są funkcje kadrowe. Administrator może dodać nowego pracownika – wprowadzając jego dane osobowe (imię, nazwisko, adres email itp.) oraz przypisując mu rolę i uprawnienia. Przy dodawaniu pracownika system generuje nowy unikalny identyfikator i tworzy dla niego kod QR. Kod ten zawiera co najmniej ID pracownika oraz datę końca ważności przepustki zgodnie z ustalonym terminem (np. koniec trwania umowy). Wygenerowany kod QR może zostać automatycznie wysłany na adres email pracownika (używając modułu FastAPI-Mail) – pracownik otrzymuje swoją przepustkę elektronicznie do wydruku lub zapisania w telefonie. Administrator może również wydrukować fizyczną kartę z tym kodem. Aktualizacja danych pracownika (np. zmiana nazwiska, przedłużenie umowy, zmiana uprawnień) jest również realizowana przez panel – administrator edytuje dane, a system może w razie potrzeby wygenerować nowy kod QR (np. gdy przedłużono datę ważności przepustki). Usunięcie pracownika spowoduje odebranie jego dostępu – jego kod QR zostaje unieważniony.

Dodatkowo, administrator ma dostęp do funkcji przeglądu raportów w panelu – może wybrać pracownika i zakres dat, aby wygenerować raport jego obecności (lista wejść oraz ewentualnych odmów). Może także wygenerować raport globalny dla całego zakładu za dany okres (np. miesiąc), aby przeanalizować ogólną efektywność systemu i przypadki nadużyć. System dba o synchronizację danych – wszystkie dane o pracownikach oraz logi zdarzeń są przechowywane w relacyjnej bazie danych i aktualizowane na bieżąco.

7. Zastosowane technologie

W projekcie wykorzystano następujące technologie i narzędzia (dobór podyktowany był wymaganiami funkcjonalnymi i środowiskowymi systemu):

- **Python 3 + FastAPI:** Backend systemu został zrealizowany w języku Python przy użyciu frameworka FastAPI. Pozwala to na szybkie prototypowanie i wysoką wydajność aplikacji webowej. FastAPI udostępnia nowoczesny mechanizm budowy API REST, automatycznie generuje dokumentację interfejsu (Swagger UI) i zapewnia walidację danych wejściowych przez modele Pydantic. Asynchroniczna architektura FastAPI umożliwia obsługę żądań w czasie rzeczywistym (co ma znaczenie przy streamingu wideo z kamery).
- **SQLAlchemy (ORM):** Do komunikacji z bazą danych wykorzystano SQLAlchemy – obiektowo-relacyjny mapper dla Pythona. Pozwala to definiować modele danych (klasy Python odpowiadające tabelom, np. Employees, Reports, Admins,

ImageFiles) i wykonywać operacje CRUD bez konieczności pisania surowych zapytań SQL.

- **SQLite:** Używany jako lokalna baza danych do przechowywania profili pracowników oraz ich embeddingów biometrycznych. Idealny dla środowisk prototypowych lub jednoosobowych stanowisk — brak wymagań co do serwera baz danych.
- **PostgreSQL:** Wykorzystywany jako zewnętrzna baza danych do przechowywania logów dostępu, raportów i wpisów operacyjnych. Zapewnia lepszą skalowalność i spójność danych w dłuższej perspektywie.
- **OpenCV (cv2):** Biblioteka używana do obsługi obrazu z kamery oraz detekcji kodów QR. W projekcie wykorzystano m.in. `cv2.VideoCapture` oraz `cv2.QRCodeDetector`, które pozwalają przechwytywać i analizować obraz w czasie rzeczywistym.
- **DeepFace:** W projekcie zastosowano bibliotekę DeepFace do biometrycznej weryfikacji tożsamości. Moduł ten pozwala generować embeddingi twarzy oraz porównywać je z zapisanymi profilami pracowników. Wbudowana funkcja `DeepFace.verify(...)` pozwala w prosty sposób określić, czy osoba próbująca wejścia została wcześniej zarejestrowana.
- **Biblioteka qrcode (Python):** Umożliwia generowanie kodów QR zawierających dane dostępu pracownika (np. ID, data ważności). Kody mogą być przekazywane jako pliki PNG lub osadzone w wiadomościach e-mail.
- **FastAPI-Mail:** Do wysyłania automatycznych e-maili (np. z przepustkami) wykorzystano bibliotekę FastAPI-Mail. Obsługuje format HTML, załączniki (np. kod QR) oraz integrację z serwerem SMTP.
- **Frontend: HTML / CSS / Bootstrap:** Interfejs administratora został zbudowany jako klasyczna aplikacja webowa z wykorzystaniem HTML, CSS i frameworka Bootstrap 5. Dzięki temu możliwa jest łatwa obsługa systemu z poziomu przeglądarki, bez potrzeby budowania pełnoprawnego SPA.

Zastosowany stos technologiczny został dobrany tak, aby system był praktyczny, prosty we wdrożeniu i jednocześnie spełniał wymagania bezpieczeństwa. FastAPI + Python zapewniają szybki rozwój backendu oraz wygodne API dla panelu administracyjnego. Warstwa danych jest rozdzielona funkcjonalnie: SQLite przechowuje dane „profilowe” (np. pracownicy i embeddingi), natomiast PostgreSQL obsługuje logi wejść/wyjść i raportowanie, co ułatwia skalowanie i analizę zdarzeń w dłuższym okresie.

Weryfikacja tożsamości opiera się na dwóch źródłach danych: identyfikacji z kodu QR (generowanego przez qrcode) oraz biometrii twarzy realizowanej przez DeepFace. Z kolei OpenCV (cv2) odpowiada za pracę ze strumieniem wideo i odczyt QR w czasie rzeczywistym. Wysyłka przepustek/komunikatów e-mail jest realizowana przez FastAPI-Mail przez serwer SMTP.

Frontend zbudowany w HTML/CSS/Bootstrap pozwala utrzymać panel administracyjny lekki i łatwy w utrzymaniu, bez komplikowania projektu frameworkiem SPA. Całość tworzy

spójny system, w którym komponenty są jasno podzielone na: przechwytywanie danych (kamera/QR), weryfikację (DeepFace), logikę biznesową (FastAPI) i trwałe przechowywanie/raportowanie (SQLite + PostgreSQL).